

1. Fundamentos

Algoritmo: pasos finitos y definidos que terminan.
Programa = algoritmo en un lenguaje. Wirth: *algoritmos + estructuras de datos = programas*.

Gramática $G = (V, T, S, P)$; $L(G) = \{w \in T^* \mid S \Rightarrow^* w\}$. Chomsky: 0 irrestricta, 1 sensible al contexto, 2 libre de contexto ($A \rightarrow a$), 3 regular ($A \rightarrow aB \mid a$). $a^n b^n$: tipo 2.

Paradigmas: imperativo (*cómo*: C, Pascal, Java) vs. declarativo (*qué*): funcional (Haskell, Lisp) y lógico (Prolog). Compilado/interpretado depende de la implementación.

Böhm-Jacopini: secuencia, selección, iteración bastan.

Métricas de módulos: *cohesión* (relación interna; alta) y *acoplamiento* (dependencia entre módulos; bajo); eficiencia: tiempo y memoria; corrección parcial/total.

MVC: controlador (coordina), modelo (datos y lógica), vista (presentación).

2. Divide y vencerás

Si P es simple $\rightarrow$ `solucion_simple(P)`; si no, descomponer en $P_1..P_k$ semejantes y menores, $S_i = \text{DyV}(P_i)$, combinar ($S1..Sk$). Termina porque el tamaño decrece.

$T(n) = aT(n/b) + f(n)$: $T(n/2) + 1 \rightarrow \log n$ (búsqueda binaria, llamar con $m \pm 1$, caso vacío *izq > der*); $2T(n/2) + 1 \rightarrow n$ (máximo, suma); $2T(n/2) + n \rightarrow n \log n$ (mergesort; QuickSort balanceado); $T(n-1) + n \rightarrow n^2$ (QuickSort peor caso); $4T(n/2) + n \rightarrow n^2$; $3T(n/2) + n \rightarrow n^{1.585}$ (Karatsuba).

QuickSort (Qsort.PAS): pivote $x = a[(l+r)\text{div } 2]$; i avanza mientras $a[i] < x$, j retrocede mientras $x < a[j]$; si $i \leq j$ intercambia; al cruzar: $\text{Sort}(l, j)$, $\text{Sort}(i, r)$. Pivote central $\neq$ partición equilibrada.
Enteros grandes: $u = Bw + x$, $v = By + z$, $uv = B^2wy + B(wz + xy) + xz$; Karatsuba: $wz + xy = (w + x)(y + z) - wy - xz$.

3. Backtracking

Árbol de soluciones parciales en profundidad: *seleccionar opción* $\rightarrow$ *Aceptable (poda)* $\rightarrow$ *anotar* $\rightarrow$ ¿completa? $\rightarrow$ *Backtracking*($i+1$) $\rightarrow$ *cancelar anotación*.

- **Una:** Hasta Éxito o NoQuedanOpciones.
- **Todos:** Hasta NoQuedanOpciones; ApuntarSoluciónCompleta.
- **Mejor:** explora todo; Si EsMejorSolución guarda; con cota $\rightarrow$ branch and bound.

Sin poda: $O(b^d)$ nodos.

8 reinas: una por fila, $\text{sol}[k] = j$; ocupados: columna j , diagonales $j-k$ y $j+k$. 92 soluciones (12 fundamentales); 4 reinas: [2,4,1,3], [3,1,4,2].

Cuadro mágico: $M_n = \frac{n(n^2+1)}{2}$: 15, 34, 65, 111. *Siamés* (n impar): 1 al centro arriba; sig. arriba-derecha con envoltura; si ocupada, abajo. 3×3 : filas 8 1 6 / 3 5 7 / 4 9 2. $n \equiv 0 \pmod{4}$: llenar $1..n^2$ y cambiar $x \mapsto n^2+1-x$ fuera de las diagonales de cada bloque 4×4 .

4. Lógica de Hoare

$\{P\} C \{Q\}$: si C inicia en P y termina, acaba en Q . **Total** = parcial + terminación (sin ciclos/recursión: parcial $\Rightarrow$ total).

Fuerza: R más fuerte que S si $R \Rightarrow S$. $\perp \Rightarrow x = 15 \Rightarrow x > 10 \Rightarrow x > 0 \Rightarrow \top$; $\{\} \equiv \top$.

Asignación: $\{Q[E/V]\} V := E \{Q\}$ (+ dominio: divisor $\neq 0$). Ej.: $\{i < 3\} i := 2i \{i < 6\}$; $\{x > 0\} x := 1/x \{x \geq 0\}$.

$$\frac{P_1 \Rightarrow P \quad \{P\} C \{Q\}}{\{P_1\} C \{Q\}} \quad \frac{\{P\} C \{Q\} \quad Q \Rightarrow Q_1}{\{P\} C \{Q_1\}}$$

$$\frac{\{P_1\} C \{Q_1\} \quad \{P_2\} C \{Q_2\}}{\{P_1 \wedge P_2\} C \{Q_1 \wedge Q_2\}} \quad (\text{igual con } \vee)$$

Composición: $\frac{\{P\} C_1 \{R\} \quad \{R\} C_2 \{Q\}}{\{P\} C_1; C_2 \{Q\}}$; se calcula *hacia atrás*: $\{x > 4\} x := x + 1; y := 2x \{y > 10\}$.

Invariante de bloque: $\{I\} C \{I\}$ (se conserva la relación). Ej. $r = 2^i$ con $i := i + 1$; $r := 2r$.

$$\frac{\{P \wedge B\} C_1 \{Q\} \quad P \wedge \neg B \Rightarrow Q}{\{P\} \text{if } B \text{ then } C_1 \{Q\}}$$

$$\frac{\{P \wedge B\} C_1 \{Q\} \quad \{P \wedge \neg B\} C_2 \{Q\}}{\{P\} \text{if } B \text{ then } C_1 \text{ else } C_2 \{Q\}}$$

Rama inalcanzable: si $P \wedge B \equiv \perp$, $\{\perp\} C \{Q\}$ vale por vacuidad ($\perp \Rightarrow$ todo).

Plantilla if: (1) escribir P, B, Q ; (2) *rama V*: $wP(C_1, Q)$ y $P \wedge B \Rightarrow wP$; (3) *rama F* con $\neg B$; (4) regla condicional; (5) sin ciclos $\Rightarrow$ total.

5. Ciclos e inducción

While B **do** C : (i) $P \Rightarrow I$; (ii) $\{I \wedge B\} C \{I\}$; (iii) $I \wedge \neg B \Rightarrow Q$; (iv) variante $V \in \mathbb{N}$ estrictamente decreciente.

Inducción: base $P(0)$ (antes de iterar) + paso $P(n) \Rightarrow P(n+1)$ (una iteración conserva).

Hallar I : trazar $n = 0..3$, escribir x_n en función de n , eliminar n , nombrar valores originales (M_0).

Programa	Invariante	Variante
Cuadrado(A)	$C = DA$	$A - D$
Potencia $R \cdot N, M-1$	$RN^M = N^{M_0}$	M
for $i < 10$: $j-$	$i+j=9$	$10-i$
suma $1..n$	$s = \frac{i(i+1)}{2}$	$n-i$
división por restas	$a = qb + r \wedge r \geq 0$	r
factorial	$f = i!$	$n-i$

6. McCarthy y Ackermann

$f(x) = x - 10$ si $x > 100$; $f(f(x+11))$ si $x \leq 100$. $f(99) = f(f(110)) = f(100) = f(f(111)) = f(101) = 91$.

Inducción modificada: $P(k) \vee, \forall n < k: [P(m) \forall m \in (n, k)] \Rightarrow P(n)$; entonces $P(n) \forall n \leq k$.

Prueba $f(x) = 91, k = 100$: base $f(100) = f(111 \rightarrow 101) = 91$. Paso ($x < 100$): (a) $x+11 > 100$: $f(x) = f(x+1) = 91$; (b) $x+11 \leq 100$: $f(x) = f(f(x+11)) = f(91) = 91$ (ambos en $(x, 100]$).

Ackermann: $A(0, y) = y + 1$; $A(x+1, 0) = A(x, 1)$; $A(x+1, y+1) = A(x, A(x+1, y))$.

$A(1, z) = z + 2$, $A(2, z) = 2z + 3$, $A(3, z) = 2^{z+3} - 3$, $A(4, z)$: torre de $z+3$ doses -3 ($A(4, 1) = 65533$). Ej. $A(2, n+1) = A(1, A(2, n)) = 2n + 5$.

Hoare: $\{x = 1\} w := A(x, y) \{w = y + 2\}$: asignación + lema + consecuencia.

7. Prolog y resolución

Regla $a :- b_1, \dots, b_m. \equiv b_1 \wedge \dots \wedge b_m \rightarrow a \equiv a \vee \neg b_1 \vee \dots \vee \neg b_m$ (Horn definida). Hecho: $a \leftarrow \top$. Meta? $-a_1, \dots, a_n \equiv \neg A_1 \vee \dots \vee \neg A_n$.

Resolución: $\frac{C \vee L \quad D \vee \neg L'}{(C \vee D)\theta}$, $\theta = \text{UMG}(L, L')$. **Re-**

futación: $S \cup \{\neg F\} \vdash \square \Rightarrow S \models F$.

Paso SLD: $\leftarrow A_1..A_k..A_n$ + regla $A \leftarrow B_1..B_m, \theta = \text{UMG}(A, A_k) \Rightarrow \leftarrow (A_1..A_{k-1}, B_1..B_m, A_{k+1}..A_n)\theta$. Éxito: $\square$. S = selección, L = lineal, D = definidas. Renombrar variables.

Unificación: $p(X, b) \sim p(a, Y)$: $\{X/a, Y/b\}$; $p(X, X) \sim p(a, b)$; $X \sim f(X)$.

Sintaxis: $;$ y; $;$ o / otra respuesta (backtracking); $\backslash +$ negación como falla; $!$ corte (elimina puntos de elección); is evalúa; $=$ unifica; $==$ compara. $X = 2+3 \rightarrow 2+3$; X is $2+3 \rightarrow 5$. Mayúscula o $_$ = variable (Maria $\neq$ maria). recibe (X) con espacio: error.

Corte verde (solo eficiencia, mayor) vs. **rojo** (cambia respuestas, max): $\text{max}(3, 5, 3)$ da true $\rightarrow$ usar $\text{max}(X, Y, Z) :- X < Y, !, Z = Y$.

```
factorial(0,1) :- !.
factorial(N,F) :- N>0, N1 is N-1,
    factorial(N1,F1), F is N*F1.
```

1. Demostrar que $\{a < b\} \text{ if } (a > b) \text{ then } m := a \text{ else } m := b \{m = b\}$ es válida

$P : a < b, B : a > b, Q : m = b$. Por la regla del **if-else** basta probar (1) $\{P \wedge B\} m := a \{Q\}$ y (2) $\{P \wedge \neg B\} m := b \{Q\}$.

(1) **Rama verdadera.** $P \wedge B \equiv a < b \wedge a > b \equiv \perp$: ningún estado la satisface, la rama es *inalcanzable*. Formalmente, por asignación $\{a = b\} m := a \{m = b\}$ (sustituir m por a en $m = b$) y, como $\perp \Rightarrow a = b$, por fortalecimiento de la precondition:

$$\{a < b \wedge a > b\} m := a \{m = b\} \text{ es válida (por vacuidad).}$$

(2) **Rama falsa.** $P \wedge \neg B \equiv a < b \wedge a \leq b \equiv a < b$. Por asignación, $\{m = b\}[b/m] \equiv b = b \equiv \top$: $\{\top\} m := b \{m = b\}$. Como $a < b \Rightarrow \top$, por consecuencia:

$$\{a < b \wedge \neg(a > b)\} m := b \{m = b\}.$$

Conclusión. Regla condicional:

$$\frac{\{a < b \wedge a > b\} m := a \{m = b\} \quad \{a < b \wedge \neg(a > b)\} m := b \{m = b\}}{\{a < b\} \text{ if } (a > b) \text{ then } m := a \text{ else } m := b \{m = b\}}$$

Sin ciclos ni recursión, termina: **corrección total**. ■

Intuición: si $a < b$ la guarda es falsa, siempre se ejecuta $m := b$. **Error a evitar:** suponer $a > b$ y tratar de obtener $m = b$; hay que reconocer que esa rama no ocurre.

2. ¿Qué métricas de módulos existen para medir su eficiencia?

Diseño modular (visto en clase):

- **Cohesión:** qué tan relacionadas están las responsabilidades internas del módulo. Se busca *alta*: fácil de nombrar, entender, probar y reutilizar.
- **Acoplamiento:** cuánto depende un módulo de otros. Se busca *bajo*: cambiar uno sin afectar a los demás.

Meta: alta cohesión y bajo acoplamiento.

Eficiencia: *tiempo de ejecución* (complejidad temporal, p.ej. $O(n \log n)$) y *uso de memoria* (complejidad espacial).

Corrección: si es correcto (parcial) y si termina (total).

Complemento: complejidad ciclomática de McCabe, líneas de código, *fan-in/fan-out*; tipos de cohesión (funcional ... coincidental) y de acoplamiento (de datos ... de contenido).

3. Algoritmo de Divide y vencerás en pseudocódigo

```

Funcion Divide_y_Vencerás( P : Problema ): Solución
{ Devuelve la solución del problema P }
Inicio
  Si P es suficientemente pequeño o simple
    Entonces devolver( solucion_simple( P ) )
  Si_No
    descomponer P en k partes( P1, P2, ..., Pk );
    Para( i = 1; i <= k; i++ )
      Si = Divide_y_Vencerás( Pi );
    devolver( combinar( S1, S2, ..., Sk ) );
Fin
  
```

Condiciones: subproblemas semejantes y de menor tamaño; caso base directo; combinación eficiente; cada llamada reduce el tamaño (terminación). Costo $T(n) = aT(n/b) + f(n)$. Ejemplos: QuickSort, búsqueda binaria, mergesort, multiplicación de enteros grandes.

4. Demostrar que el invariante es $i + j = 9$

```

int j = 9;
for( int i=0; i<10; i++ ) j--;
  
```

Equivale a $j:=9; i:=0; \text{ while } i<10 \text{ do } \{ j:=j-1; i:=i+1 \}$. Sean i_n, j_n los valores tras n iteraciones; traza $(0,9), (1,8), (2,7), \dots$ $P(n): i_n + j_n = 9$.

Caso base ($n=0$): $i_0 = 0, j_0 = 9, i_0 + j_0 = 9$. ✓

Hipótesis de inducción: $i_n + j_n = 9$.

Paso inductivo: si $i_n < 10$, el cuerpo da $j_{n+1} = j_n - 1, i_{n+1} = i_n + 1$:

$$i_{n+1} + j_{n+1} = (i_n + 1) + (j_n - 1) = i_n + j_n \stackrel{\text{H.I.}}{=} 9.$$

Por inducción, $i + j = 9$ se cumple tras cualquier número de iteraciones: es invariante. ■

Forma Hoare: $\{i+j=9 \wedge i<10\} j:=j-1; i:=i+1 \{i+j=9\}$: hacia atrás, por $i:=i+1: i+1+j=9$; por $j:=j-1: i+j=9$. ✓

Salida: también $0 \leq i \leq 10$ es invariante; al salir $\neg(i < 10) \Rightarrow i = 10$ y $j = 9 - 10 = -1$.

Terminación: variante $10 - i$ (entera, ≥ 0 , baja 1 por vuelta): 10 iteraciones, corrección total.

5. Backtracking para buscar la mejor solución

Construye la solución por **etapas** i : prueba cada opción, anota solo las *aceptables* (poda), avanza a $i+1$ y al volver cancela la anotación para probar la siguiente (vuelta atrás). Recorre en profundidad el árbol de soluciones parciales.

```

Funcion Backtracking( Etapa i, solucionMejor ) devuelve: boolean
Inicio
  Éxito = falso;
  IniciarOpciones( i, GrupoOpciones o );
  Repetir
    SeleccionarNuevaOpcion( o, Opcion n );
    Si ( Aceptable( n ) ) entonces
      AnotarOpcion( i, n );
      Si SolucionCompleta( i ) entonces
        Éxito = verdadero;
        Si EsMejorSolucion( solucionActual, solucionMejor ) entonces
          ApuntarSolucionActual(); // solucionMejor := actual
        finsi;
      Sino
        Éxito = Backtracking( i+1, solucionMejor );
        Si Éxito = falso entonces cancelamosAnotacion( i, n ); finsi;
      Finsi;
    Finsi;
  Hasta ( NoQuedanOpciones( o ) ); // no para al primer éxito
  Retorna Éxito;
Fin
  
```

Diferencias: *una solución* termina con Hasta (Éxito = verdadero) o NoQuedanOpciones; *todas* recorre todo y guarda cada una (ApuntarSolucionCompleta); *la mejor* recorre todo y conserva solo la mejor según un **criterio de calidad** (EsMejorSolucion). Si se podan ramas cuya cota no puede superar a la mejor actual: *branch and bound*. Para seguir explorando tras registrar una solución hay que deshacer la anotación. Sin poda: $O(b^d)$.